

[Задание 1 \(подготовительное\)](#)[Задание 2 \(основное\)](#)[Задание 3 \(проверяющее\)](#)

# Практика 7. Полиморфизм подтипов (I)

## Задание 1 (подготовительное)

Предоставьте интерфейс `Serializable` для (де)сериализации объекта в универсальное представление.

В качестве универсального представления используйте следующий тип `Value`, являющийся абстракцией над типами `JSON`:

```
struct Value;

using ArrayValue = std::vector<Value>;
using ObjectValue = /* См. примечание */;

struct Value {
    std::variant<std::nullptr_t, bool, double, std::string, ArrayValue, ObjectValue> v;

    Value(std::nullptr_t) : v(nullptr) {}
    Value(bool v) : v(v) {}
    Value(double v) : v(v) {}
    Value(std::string v) : v(std::move(v)) {}
    Value(ArrayValue v) : v(std::move(v)) {}
    Value(ObjectValue v) : v(std::move(v)) {}
};
```

### Примечание.

В качестве псевдонима `ObjectValue` остановиться на любом из трёх вариантов:

- `std::map<std::string, Value>;`
- `std::unordered_map<std::string, Value>;`
- `std::vector<std::pair<std::string, Value>>.`

Вопросы для размышления:

- Каковы недостатки и преимущества каждого из вариантов?

- Какой из вариантов соответствует спецификации JSON?

## Задание 2 (основное)

Предоставьте интерфейс `Serializer` для сохранения (загрузки) универсального представления (значения типа `Value`) в (из) конкретную байтовую последовательность определённого формата.

Предоставьте следующие реализации интерфейса:

- `JsonSerializer` — для работы с JSON-форматом.

Реализовать только сохранение (загрузка — в качестве усложнённого факультатива).

- `XmlSerializer` — для работы с XML-форматом.

Реализовать только сохранение (загрузка — в качестве усложнённого факультатива).

- `BinarySerializer` — произвольный (свой) бинарный формат.

## Задание 3 (проверяющее)

- Предоставьте несколько тестовых классов/структур, реализующих интерфейс `Serializable`
- Проверьте работу реализованных сериализаторов.